

Guide Playwright

Tout ce qu'il faut pour **comprendre**, **lancer** et **présenter** les tests E2E du projet Hublot.

Fichiers concernés : `playwright.config.ts` , dossier `e2e/`

Lien plan de test : [PLAN_TEST.md](./1.3 Rédiger des scripts dans la démarche DevOps/1.3.7 Automatiser les tests en DevOps.md) · [SCENARIOS_TEST.md](./1.2 Préparer le déploiement d'une application/1.2.2 Elaborer un scénario de test.md) · [RAPPORT_EXECUTION_TESTS.md](./1.2 Préparer le déploiement d'une application/1.2.9 Rapport d'exécution des tests.md)

Démarrage rapide

Objectif	Commande
Ouvrir l'interface graphique Playwright (choisir un fichier de test, lancer / rejouer, timeline des actions)	<code>npm run test:e2e:ui</code>
Démo jury : navigateur visible + ralenti sur le parcours smoke	<code>npm run test:e2e:demo</code>
Vérif locale rapide sans fenêtre (comme en CI)	<code>npm run test:e2e</code>

L'UI s'ouvre dans le navigateur après la commande `test:e2e:ui` ; le détail est en §7 (Mode interface).

1. C'est quoi Playwright ?

Playwright (Microsoft, open source) **pilote un vrai navigateur** (Chromium, Firefox, Safari) pour simuler un utilisateur :

- ouvrir une URL ;
- remplir des champs, cliquer ;
- vérifier ce qui s'affiche.

Outil	Rôle dans Hublot
Vitest	Tests unitaires (calculs, filtres) — pas de navigateur

Outil	Rôle dans Hublot
Playwright	Tests E2E (end-to-end) — parcours complet dans le navigateur

En **CI** (GitHub Actions), Playwright tourne **sans fenêtre** (headless) : rapide, adapté au serveur.

En **local**, tu peux **voir** le navigateur pour la démo jury.

2. Prérequis (une seule fois)

```
cd /chemin/vers/StatDirm
npm ci
npx playwright install chromium
```

`playwright install` télécharge un Chromium dédié aux tests (séparé de ton Chrome habituel).

3. Comment ça marche dans ce projet

1. Playwright **build** l'app en mode test : `npm run build:e2e` (identifiants `e2e-user` / `e2e-pass`, badge « Test local (QA) »).
2. Il **démarre** un serveur preview sur <http://127.0.0.1:4173>.
3. Il **exécute** les fichiers `e2e/*.spec.ts`.
4. À la fin, le serveur s'arrête (sauf en mode UI).

Configuration : `playwright.config.ts`.

4. Commandes — laquelle utiliser ?

Commande	Fenêtre visible ?	Vitesse	Usage
<code>npm run test:e2e</code>	Non (headless)	Rapide	CI, vérif locale
<code>npm run test:e2e:demo</code>	Oui	Ralenti (~800 ms)	Démo jury (recommandé)

Commande	Fenêtre visible ?	Vitesse	Usage
<code>npm run test:e2e:slow</code>	Oui	Ralenti (~400 ms)	Démo un peu plus vive
<code>npm run test:e2e:headed</code>	Oui	Rapide	Voir sans ralentir
<code>npm run test:e2e:ui</code>	Oui (UI Playwright)	Au choix	Interface graphique : un test à la fois, replay, pas de <code>--debug</code>
<code>npm run test:campaign</code>	Non	Rapide	Campagne ST-F01... (tous les scénarios doc)

À retenir : pour l'UI → `npm run test:e2e:ui` ; pour la **démo scénarisée au jury** → `npm run test:e2e:demo` (voir aussi le tableau Démarrage rapide en tête de ce guide).

Commande star pour la démo (timing validé)

```
npm run test:e2e:demo
```

Équivalent manuel :

```
PLAYWRIGHT_SLOW_MS=800 npm run test:e2e:headed -- e2e/smoke.spec.ts
```

Un seul test (connexion uniquement) :

```
PLAYWRIGHT_SLOW_MS=800 npm run test:e2e:headed -- e2e/smoke.spec.ts -g "connexion"
```

5. Scénario `e2e/smoke.spec.ts` (3 tests)

#	Nom	Ce que le jury voit / ce qui est vérifié
1	Page de connexion	Titre « Bienvenue », champs email / mot de passe, bouton Se connecter

#	Nom	Ce que le jury voit / ce qui est vérifié
2	Connexion → tableau de bord	Saisie <code>e2e-user</code> / <code>e2e-pass</code> , clic, bouton Déconnexion visible
3	<code>health.json</code>	API interne : <code>{ "status": "ok", "app": "hublot" }</code> (sans ouvrir le navigateur)

Identifiants **uniquement** pour le build de test (`build:e2e`), pas les secrets Netlify prod.

6. Déroulé oral suggéré (3-5 min)

Avant de commencer

- Terminal ouvert à la racine du projet.
- Fermer les onglets inutiles (le build initial peut prendre **15-30 s** la première fois).

Script

1. Intro (30 s)

« Playwright automatise le navigateur. En CI c'est headless ; ici je montre le parcours critique : login puis accès au tableau de bord. »

2. Lancer la démo

```
npm run test:e2e:demo
```

Alternative — interface Playwright : `npm run test:e2e:ui` , puis lancer `smoke.spec.ts` depuis l'UI (utile pour expliquer test par test).

Pendant l'exécution, commenter :

- ouverture de la page login ;
- remplissage des champs ;
- apparition du tableau de bord et du bouton Déconnexion.

3. Résultat terminal

Montrer les **3 passed** (ou 2 visibles + 1 test API).

4. Lien DevOps (30 s)

« Le même type de test tourne dans le workflow **CI** sur GitHub Actions à chaque push. »

5. Optionnel — prod sans login

```
curl -sI https://dirmhublot.netlify.app | grep -i x-frame
```

Headers de sécurité (ST-SEC01).

7. Mode interface (`test:e2e:ui`)

Commande pour ouvrir l'interface graphique :

Pour **choisir un test**, le relancer, ou avancer pas à pas :

```
npm run test:e2e:ui
```

1. Une fenêtre **Playwright UI** s'ouvre dans le navigateur.
2. Cliquer sur `smoke.spec.ts` → lancer un test (▶).
3. Tu vois la timeline des actions et peux **watch** le replay.

Idéal si tu veux expliquer **ligne par ligne** sans `--debug` .

8. Pièges fréquents

« Ça va trop vite »

→ Utilise `npm run test:e2e:demo` (800 ms entre les actions) ou augmente :

```
PLAYWRIGHT_SLOW_MS=1200 npm run test:e2e:headed -- e2e/smoke.spec.ts
```

`--debug` + **page blanche** `about:blank`

Normal : le mode **debug met en pause** avant chaque action.

Il faut cliquer **Resume (▶)** dans l'inspecteur Playwright.

Pour une démo fluide, **ne pas utiliser** `--debug` — préférer `test:e2e:demo` ou `test:e2e:ui` .

« Rien ne s'affiche » au tout début

Le **premier** lancement fait `build:e2e` + preview : **attendre** jusqu'à ce que Chromium s'ouvre (peut prendre ~20 s).

Port 4173 déjà utilisé

```
lsof -i :4173 # voir le processus
# ou fermer l'autre preview / relancer le terminal
```

Tests sur la vraie prod

`smoke.spec.ts` cible **127.0.0.1:4173** (preview locale).
Pour la prod en lecture seule, voir `e2e/campaign-prod.spec.ts` et `npm run test:campaign`.

9. CI vs local

	Local (démon)	GitHub Actions (CI)
Fenêtre	Visible avec <code>test:e2e:demo</code>	Headless
URL	http://127.0.0.1:4173	Idem (sur le runner)
Commande CI	—	<code>npm run test:e2e</code> dans <code>.github/workflows/ci.yml</code>
Preuve	Terminal + option capture vidéo	Onglet Actions → job E2E vert

10. Ajuster le ralenti

Variable d'environnement `PLAYWRIGHT_SLOW_MS` (millisecondes entre chaque action) :

Valeur	Effet
<code>0</code> (défaut)	Vitesse normale
<code>400</code>	<code>npm run test:e2e:slow</code>

Valeur	Effet
800	<code>npm run test:e2e:demo</code> (timing retenu pour la démo)
1200	Très lent (grand public / capture vidéo)

11. Fichiers du projet

```
e2e/  
  smoke.spec.ts           # 3 tests – parcours critique (démo)  
  campaign-manual.spec.ts # ST-F01...F06 (campagne doc)  
  campaign-prod.spec.ts   # vérifs prod lecture seule  
  playwright.config.ts    # port 4173, webServer, slowMo
```

12. Checklist avant soutenance

- ☐ `npm ci` et `npx playwright install chromium` faits
- ☐ `npm run test:e2e:demo` testé une fois (3 passed)
- ☐ Savoir expliquer : Vitest vs Playwright, headless vs headed
- ☐ Onglet GitHub Actions **CI** ouvert (job E2E)
- ☐ (Optionnel) `npm run test:e2e:ui` — ouvrir l'UI Playwright et savoir lancer `smoke.spec.ts` depuis l'interface

13. Phrase de conclusion jury

« Playwright rejoue automatiquement le parcours utilisateur critique. En local je le montre en ralenti avec `test:e2e:demo` ; en intégration continue, les mêmes tests tournent headless et bloquent un déploiement si la connexion ou le build de test régressent. »